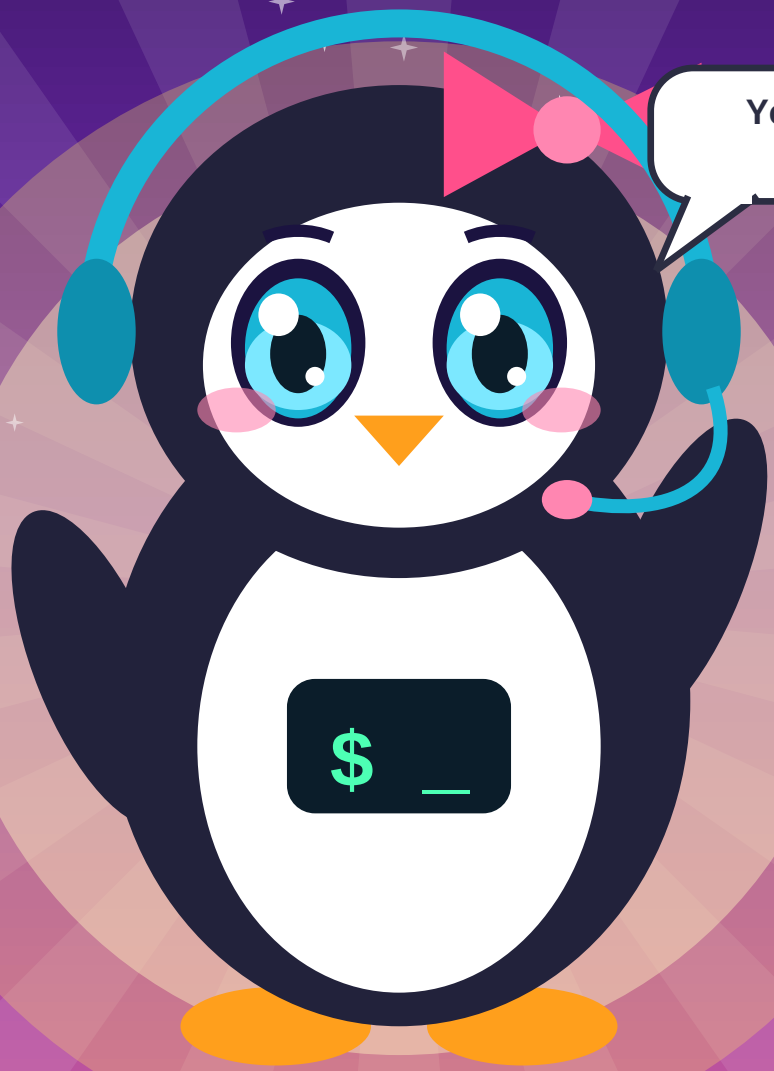
# INTERVIEW BOSS BATTLES II

20 NEW SCENARIOS | KUBERNETES | AWS | TERRAFORM | CI/CD




FINAL BOSS: THE SENIOR INTERVIEWER HP 100%

- CREATED BY -
RUTUJA NAIK

SET 2 * 20 SCENARIOS * 10 RAPID-FIRE

## 00 HOW TO ANSWER

*TRACE again, plus trade-off thinking*

| | |
|---|---|
| **T** | **Triage**<br>How bad is it, who is affected, since when, and what changed recently? |
| **R** | **Read the evidence**<br>Check logs, metrics and state before touching anything. |
| **A** | **Analyze by layer**<br>Narrow it down: CPU, memory, disk, network, then application and dependencies. |
| **C** | **Change safely**<br>Smallest fix first, one change at a time, with a rollback ready. |
| **E** | **Educate and prevent**<br>Monitoring, automation, runbook and a blameless post-mortem. |

### YOUR 60-90 SECOND ANSWER TEMPLATE

**1.** Restate the problem and the impact (10 s). **2.** Say your order of hypotheses (10 s). **3.** Walk through 3 to 5 commands and what each tells you (40 s). **4.** State the fix and your rollback plan (15 s). **5.** Finish with prevention: monitoring, automation, documentation (10 s).

### POWER PHRASES THAT SOUND SENIOR

"I'd start by clarifying requirements and constraints."
"Let me think through the trade-offs of each option."
"I'd start with the simplest check, then go deeper layer by layer."
"I'd mitigate first, find the root cause, then add automation and monitoring."
"I don't know that one offhand, but here is how I would find out."

### QUEST LOG (CONTENTS)


| | | | | | |
|---|---|---|---|---|---|
| 01 | Kubernetes node is NotReady | 2 | 11 | Clock drift breaks authentication | 12 |
| 02 | Service unreachable inside the cluster | 3 | 12 | Container cannot reach the internet | 13 |
| 03 | Intermittent DNS timeouts in Kubernetes | 4 | 13 | Huge Docker image and slow builds | 14 |
| 04 | ImagePullBackOff in production | 5 | 14 | CI pipeline is slow and flaky | 15 |
| 05 | EC2 instance failing status checks | 6 | 15 | Terraform state lock or lost state | 16 |
| 06 | S3 AccessDenied mystery | 7 | 16 | A secret leaked into Git | 17 |
| 07 | Load balancer targets are unhealthy | 8 | 17 | AWS bill suddenly doubled | 18 |
| 08 | Server drops into emergency mode | 9 | 18 | Patch 500 servers safely | 19 |
| 09 | High CPU but top shows nothing | 10 | 19 | Backup and disaster recovery plan | 20 |
| 10 | Database: too many connections | 11 | 20 | Design a highly available web app | 21 |

**Troubleshooting** | **Admin and security** | **DevOps and cloud** | then: rapid-fire round and final checklist

# 01 KUBERNETES NODE IS NOTREADY

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**In your Kubernetes cluster one node shows NotReady, and pods on it are stuck or being evicted. What do you do?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

NotReady means the control plane isn't getting healthy heartbeats from the kubelet on that node. I'd start with `kubectl describe node` and read the Conditions and Events: it tells me whether it's memory, disk or PID pressure, or simply that the kubelet stopped reporting. Then I get onto the node itself, via SSH or SSM, and check `systemctl status kubelet`, `journalctl -u kubelet` and the container runtime, usually containerd. Common causes are a full disk, out of memory, a crashed kubelet or runtime, CNI or network problems, or an expired certificate. If I can't fix it quickly, I `cordon` and `drain` the node so workloads move to healthy nodes, then replace the node instead of nursing it, especially in autoscaling node groups.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `kubectl get nodes -o wide` | Which nodes are NotReady, and since when? |
| 2 | `kubectl describe node NODE` | Conditions (MemoryPressure, DiskPressure, PIDPressure, Ready), taints and recent Events. |
| 3 | `systemctl status kubelet (on the node)` | Is the kubelet running or crash-looping? |
| 4 | `journalctl -u kubelet -n 100 --no-pager` | The real error: certificate expiry, CNI not ready, runtime unreachable, cgroup problems. |
| 5 | `systemctl status containerd and crictl ps` | Is the container runtime healthy, and are containers running? |
| 6 | `df -h and df -i and free -m` | Disk, inode and memory pressure on the node. |
| 7 | `kubectl cordon NODE; kubectl drain NODE --ignore-daemonsets --delete-emptydir-data` | Stop new pods and move existing ones away safely. Check PodDisruptionBudgets first. |
| 8 | Cloud: check instance status checks, or replace the node | In managed node groups, terminate the node and the group creates a fresh one. |

## COMMON ROOT CAUSES

Kubelet or runtime crashed, disk full (images or logs), memory exhaustion, CNI or network failure, expired kubelet certificate, node VM unhealthy or rebooted, cloud instance failing status checks.

## FIX AND PREVENTION

Fix the underlying cause (free disk, restart kubelet, repair network) or replace the node. Prevent it with node monitoring (node_exporter), disk alerts, image garbage-collection thresholds, node groups with auto-repair, and PodDisruptionBudgets.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. What happens to pods on a NotReady node?**

A. After the toleration period (5 minutes by default) pods are evicted and rescheduled. StatefulSet pods can stay stuck until the node is confirmed gone.

**Q. cordon versus drain?**

A. Cordon marks the node unschedulable. Drain also evicts the running pods, respecting PodDisruptionBudgets.

## AVOID THIS MISTAKE

Deleting pods or the node object without finding why. The same fault will hit the next node.

# 02 SERVICE UNREACHABLE INSIDE THE CLUSTER

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**A pod cannot reach another service in the cluster using its Service name. How do you debug it?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

I follow the path a request takes: DNS name, Service, endpoints, pod, then network policy. First I run a debug pod and check whether the name resolves with `nslookup`. If DNS works, I check whether the Service actually has endpoints with `kubectl get endpoints`. An empty list almost always means the Service selector doesn't match the pod labels, or the pods aren't Ready. Then I test the pod directly by IP and port, to separate Service problems from application problems, and verify `port` versus `targetPort`. Finally I check NetworkPolicies, because a default-deny policy silently drops traffic, and the health of kube-proxy and the CNI.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `kubectl run tmp --rm -it --image=busybox:1.36 -- sh` | Throwaway pod in the same namespace to test from the inside. |
| 2 | `nslookup my-svc then nslookup my-svc.ns.svc.cluster.local` | Does the name resolve to the ClusterIP? Try the full name too. |
| 3 | `kubectl get svc my-svc -o wide` | ClusterIP, port, targetPort and the selector. |
| 4 | `kubectl get endpoints my-svc` | Empty endpoints means selector mismatch or no Ready pods. |
| 5 | `kubectl get pods --show-labels -l app=web` | Do the pod labels match the Service selector exactly? |
| 6 | `wget -qO- http://POD_IP:8080/health` | Test the pod directly. If this works, the problem is in the Service layer. |
| 7 | `kubectl get networkpolicy -n ns` | A restrictive policy may block ingress or egress. |
| 8 | `kubectl -n kube-system get pods -l k8s-app=kube-dns` | Is CoreDNS running? Check kube-proxy and the CNI pods too. |

## COMMON ROOT CAUSES

Selector or label mismatch, pods not Ready (failing readiness probe), wrong targetPort, NetworkPolicy blocking traffic, wrong namespace in the DNS name, CoreDNS or kube-proxy problems, app listening on 127.0.0.1 only.

## FIX AND PREVENTION

Fix labels or selector, the readiness probe or ports, adjust the NetworkPolicy, and make the app listen on 0.0.0.0. Use readiness-gated rollouts so broken pods never receive traffic.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Service port versus targetPort?**

A. port is what clients use on the Service. targetPort is the container port the traffic is forwarded to.

**Q. ClusterIP, NodePort, LoadBalancer?**

A. ClusterIP: internal only. NodePort: opens a port on every node. LoadBalancer: asks the cloud for an external load balancer.

## AVOID THIS MISTAKE

Assuming DNS is broken when the real problem is empty endpoints.

# 03 INTERMITTENT DNS TIMEOUTS IN KUBERNETES

TROUBLESHOOTING DIFFICULTY ✦✦✦

**THE QUESTION**

**Applications in your cluster randomly fail with DNS timeouts or 'Temporary failure in name resolution'. How do you investigate?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

Intermittent DNS in Kubernetes usually comes from load on CoreDNS, the `ndots` setting, or conntrack races with UDP. I'd check CoreDNS pod health, restarts and CPU, then its logs and metrics. Then I look at how the application resolves names: with the default `ndots:5`, a name like `api.example.com` triggers several failed search-domain lookups before the real one, multiplying DNS traffic. I'd test resolution from a pod repeatedly with timing, and check the node's conntrack counters for dropped packets. Typical fixes are scaling CoreDNS, enabling NodeLocal DNSCache, using fully qualified names with a trailing dot, lowering `ndots` in the pod's dnsConfig, and fixing upstream resolver limits.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `kubectl -n kube-system get pods -l k8s-app=kube-dns -o wide` | Are CoreDNS pods healthy, spread across nodes and not restarting? |
| 2 | `kubectl -n kube-system top pods -l k8s-app=kube-dns` | Is CoreDNS overloaded or CPU-throttled? |
| 3 | `kubectl -n kube-system logs -l k8s-app=kube-dns --tail=50` | Upstream timeouts, loops or SERVFAIL errors. |
| 4 | `kubectl exec pod -- cat /etc/resolv.conf` | See the search domains and `options ndots:5`. |
| 5 | `for i in $(seq 1 50); do time nslookup example.com; done` | Repeat a lookup from a pod to see how often it is slow or fails. |
| 6 | `dig @CLUSTER_DNS_IP example.com +stats` | Query CoreDNS directly and read the query time. |
| 7 | `conntrack -S (on the node)` | insert_failed or drop counters point to conntrack race conditions. |
| 8 | `kubectl get cm coredns -n kube-system -o yaml` | The Corefile: forwarders, cache and loop plugin settings. |

**COMMON ROOT CAUSES**

CoreDNS under-provisioned, ndots:5 search-path amplification, conntrack races with UDP DNS, upstream resolver rate limits (cloud VPC resolver), a network policy blocking DNS, a DNS forwarding loop.

**FIX AND PREVENTION**

Scale CoreDNS (or use the DNS autoscaler), deploy NodeLocal DNSCache, lower `ndots` through `dnsConfig` (for example 2), use FQDNs with a trailing dot, and enable caching. Alert on DNS latency and SERVFAIL rate.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. What does ndots:5 do?**

A. A name with fewer than 5 dots is tried with each search domain first, so external names cause several useless lookups.

**Q. What is NodeLocal DNSCache?**

A. A DNS cache DaemonSet on each node, so pods query locally and avoid conntrack and CoreDNS hot spots.

**AVOID THIS MISTAKE**

Blaming the application first. Measure DNS from a pod and look at CoreDNS.

# 04 IMAGEPULLBACKOFF IN PRODUCTION

TROUBLESHOOTING DIFFICULTY ✦ ✦ ✦

**THE QUESTION**

**After a deployment, pods are stuck in ImagePullBackOff. How do you find and fix the cause?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

ImagePullBackOff means the kubelet can't download the container image and is backing off between retries. The exact reason is in the pod's Events from `kubectl describe pod`: 'not found' means a wrong image name or tag, 'unauthorized' means missing or wrong registry credentials, 'timeout' means a network problem, and 'no matching manifest' means the image doesn't exist for that CPU architecture. I'd verify the image reference first, then check that the imagePullSecret exists in the same namespace and is referenced by the pod or its ServiceAccount. For ECR, tokens expire after 12 hours, so I check how credentials are refreshed. I'd also try pulling the image from the node itself with `crictl pull`.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `kubectl describe pod p` | Events show the real error: not found, unauthorized, timeout or no matching manifest. |
| 2 | `kubectl get pod p -o jsonpath='{.spec.containers[*].image}'` | The exact image and tag the pod uses. |
| 3 | `kubectl get secret regcred -o yaml` | Does the pull secret exist in this namespace, with type dockerconfigjson? |
| 4 | `kubectl get pod p -o yaml \| grep -A3 imagePullSecrets` | Is the secret referenced by the pod or its ServiceAccount? |
| 5 | `crictl pull IMAGE (on the node)` | Reproduce the pull from the node with the container runtime. |
| 6 | `curl -I https://registry.example.com/v2/` | Can the node reach the registry (NAT, firewall, DNS, proxy)? |
| 7 | `docker manifest inspect IMAGE` | Does the tag exist, and for the right architecture (amd64 or arm64)? |
| 8 | ECR: aws ecr get-login-password | Tokens expire after 12 hours. Check how nodes or secrets get refreshed. |

**COMMON ROOT CAUSES**

Typo in the image or tag, tag deleted or never pushed, private registry without credentials, expired token, Docker Hub rate limit, registry unreachable from private subnets, wrong CPU architecture.

**FIX AND PREVENTION**

Fix the reference or credentials, use immutable version tags or digests, mirror images into a private registry, give nodes or service accounts proper registry permissions, and set `imagePullPolicy` deliberately.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. Why avoid the :latest tag?**

A. It is mutable, so nodes can run different images and rollbacks are unclear. Use version tags or digests.

**Q. What is imagePullPolicy?**

A. Always, IfNotPresent or Never. It decides when the kubelet pulls the image.

**AVOID THIS MISTAKE**

Using :latest in production, and not checking the Events first.

# 05 EC2 INSTANCE FAILING STATUS CHECKS

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

**An EC2 instance shows as running, but you cannot reach it and it is failing status checks. What do you do?**

SAY IT LIKE THIS (60-90 seconds, out loud)

AWS has two status checks, and they tell me which layer is broken. A system status check failure is a problem with the underlying host, so I stop and start the instance to move it to new hardware, or let auto-recovery do it. An instance status check failure means the operating system itself has an issue: a kernel panic, a bad fstab, exhausted memory or broken networking. I'd read the console output and screenshot from EC2, which need no network access, to see boot errors. If a disk configuration is broken, I stop the instance, detach the root volume, attach it to a rescue instance, fix the files and reattach it. Before touching anything, I take a snapshot.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `aws ec2 describe-instance-status --instance-ids i-123` | Which check fails: system or instance? |
| 2 | `aws ec2 get-console-output --instance-id i-123 --latest` | Serial console log: kernel panic, fstab or network errors. |
| 3 | EC2 console: Monitor and troubleshoot, Get screenshot | See what the instance displays, for example emergency mode. |
| 4 | `aws ec2 describe-security-groups and describe-network-acls` | Is inbound traffic allowed? NACLs are stateless, so check both directions. |
| 5 | `aws ssm start-session --target i-123` | If the SSM agent works, you get a shell without SSH. |
| 6 | `aws ec2 create-snapshot --volume-id vol-123` | Snapshot before any repair work. |
| 7 | Rescue flow: stop, detach root volume, attach to helper instance | Mount it, fix /etc/fstab, SSH keys or network config, then reattach with the original device name. |
| 8 | `aws ec2 stop-instances then start-instances` | Stop and start moves a system-check failure to new host hardware. |

COMMON ROOT CAUSES

Failed host (system check), kernel panic or bad fstab entry (instance check), out of memory or disk, misconfigured networking, exhausted CPU credits on burstable instances, wrong security group or NACL.

FIX AND PREVENTION

Repair using a rescue instance or the serial console. Then add CloudWatch alarms with auto-recover actions, use `nofail` in fstab for non-critical mounts, and keep AMIs and snapshots current.

FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Reboot versus stop and start?**

A. Reboot keeps the same host. Stop and start usually moves to new hardware, changes the public IP unless you use an Elastic IP, and wipes instance-store data.

**Q. NACL versus security group?**

A. Security groups are stateful, allow-only, and apply to instances. NACLs are stateless, support deny rules, and apply to subnets.

AVOID THIS MISTAKE

Terminating or stopping before taking a snapshot.

# 06 S3 ACCESSDENIED MYSTERY

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**An application gets AccessDenied reading an S3 object, even though you believe its role has access. How do you troubleshoot?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

AccessDenied means at least one policy layer says no, and an explicit deny always wins. First I identify exactly who is calling, with `aws sts get-caller-identity`, because it's often the wrong role or profile. Then I check each layer: the IAM identity policy, the bucket policy, S3 Block Public Access, any permissions boundary or SCP from AWS Organizations, the KMS key policy if the objects are encrypted, and VPC endpoint policies. For cross-account access, both sides must allow it. The IAM policy simulator and CloudTrail show which request was denied. I'd then fix the narrowest policy with least privilege, rather than granting `s3:*`.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `aws sts get-caller-identity` | Who am I really? Wrong profile or role is the most common cause. |
| 2 | `aws s3api get-bucket-policy --bucket b` | Any explicit Deny, or a missing Allow for this principal? |
| 3 | `aws iam list-attached-role-policies --role-name r` | Identity policies attached to the role. |
| 4 | `aws iam simulate-principal-policy --policy-source-arn ARN --action-names s3:GetObject --resource-arns arn:aws:s3:::b/*` | Simulate whether the action is allowed. |
| 5 | `aws s3api get-public-access-block --bucket b` | Block Public Access settings that override policies. |
| 6 | `aws kms get-key-policy --key-id K --policy-name default` | Encrypted objects also need kms:Decrypt on the key. |
| 7 | Organizations: SCPs and permissions boundaries | They limit what even an admin can do. Look for explicit denies. |
| 8 | CloudTrail: find the AccessDenied event | Shows the action, resource and principal that were denied. |

## COMMON ROOT CAUSES

Wrong credentials or role, missing Allow, explicit Deny in the bucket policy, missing KMS permissions, SCP or boundary limiting access, VPC endpoint policy, cross-account setup missing one side, object owned by another account.

## FIX AND PREVENTION

Grant least-privilege access exactly where needed (action and resource), fix the KMS key policy, and test with the simulator. Manage IAM and bucket policies as code to avoid drift.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. What is the IAM evaluation logic?**

A. An explicit deny wins. Otherwise an allow is required, else it is an implicit deny. Cross-account access needs allows on both sides.

**Q. IAM role versus IAM user?**

A. Roles provide temporary credentials and are assumed by services or users. Prefer roles over long-lived access keys.

## AVOID THIS MISTAKE

Fixing it with s3:* on *. That creates a security risk.

# 07 LOAD BALANCER TARGETS ARE UNHEALTHY

DEVOPS & CLOUD DIFFICULTY ✦✦✦

**THE QUESTION**

**Your load balancer shows all targets as unhealthy and users get 502 or 503 errors. What do you check?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

If targets are unhealthy, the load balancer isn't getting the expected response from its health check, so it stops sending traffic. I'd check the target group's health check settings first: protocol, port, path and success codes, plus the reason shown for each target. Then I run that exact request from inside the VPC with `curl`, because it might fail on the instance itself, like the app listening only on localhost, or a redirect returning 301 where 200 is expected. I'd verify security groups allow the load balancer to reach the target port, and look in the app logs for the health check requests. I'd also check whether the health endpoint depends on something that is down, like the database.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `aws elbv2 describe-target-health --target-group-arn ARN` | Each target's state and reason, such as Target.Timeout or Target.ResponseCodeMismatch. |
| 2 | `aws elbv2 describe-target-groups --target-group-arns ARN` | Health check path, port, interval, thresholds and matcher. |
| 3 | `curl -v http://TARGET_IP:PORT/health` | Run the same request from inside the VPC. |
| 4 | `ss -tulpn \| grep :PORT (on the target)` | Is the app listening on 0.0.0.0, not just 127.0.0.1? |
| 5 | Security groups | The target's SG must allow the port from the load balancer's SG. |
| 6 | `tail -f /var/log/nginx/access.log` | Do health check requests arrive, and what status do they get? |
| 7 | `kubectl get endpoints and kubectl describe ingress` | On Kubernetes: do endpoints exist, and is the readiness probe passing? |
| 8 | Health endpoint dependencies | An endpoint that calls a down database marks every target unhealthy. |

**COMMON ROOT CAUSES**

Wrong health check path or port, the app returns 301, 401 or 500 on that path, security group blocks the load balancer, app bound to localhost, slow startup exceeding the check, health check depending on a failing dependency, timeouts too strict.

**FIX AND PREVENTION**

Provide a lightweight `/health` that returns 200, correct the port and success codes, open the security group, tune thresholds and timeouts, and use a deregistration delay so deployments don't drop requests.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. Liveness versus readiness versus LB health checks?**

A. Liveness restarts a container. Readiness gates traffic inside Kubernetes. LB health checks gate traffic at the cloud edge. Keep them consistent.

**Q. 502 versus 504 on an ALB?**

A. 502: invalid or no response from the target. 504: the target didn't answer within the idle timeout.

**AVOID THIS MISTAKE**

A health check that tests every dependency. One outage then removes all targets.

# 08 SERVER DROPS INTO EMERGENCY MODE

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**After a reboot, a Linux server boots into emergency mode and asks for the root password. What happened and how do you fix it?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

Emergency mode at boot almost always means systemd couldn't mount a filesystem it considers essential, most often a bad entry in `/etc/fstab` after a disk change, a typo or a missing UUID. I'd log in with the root password or console and look at `journalctl -xb` and `systemctl --failed` to see which mount failed. I compare the UUIDs in fstab with `blkid`, fix or comment out the bad line, or add `nofail` for non-critical mounts, and validate with `findmnt --verify` and `mount -a` before rebooting. If the root filesystem is damaged, I boot from rescue media, run `fsck` on the unmounted filesystem, and chroot to repair the bootloader if required.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `journalctl -xb` | Boot log with explanations: which mount or service failed. |
| 2 | `systemctl --failed` | Failed units. Look for `*.mount` units. |
| 3 | `cat /etc/fstab and blkid` | Compare the UUIDs and device names. |
| 4 | `mount -o remount,rw /` | Make root writable so you can edit files (emergency mode mounts it read-only). |
| 5 | `vi /etc/fstab` | Fix the UUID, or add `nofail` (and `x-systemd.device-timeout=5s`) for non-critical mounts. |
| 6 | `findmnt --verify and mount -a` | Validate the file and test mounting before you reboot. |
| 7 | `fsck -f /dev/sdX1 (unmounted)` | Repair a damaged filesystem from rescue mode. |
| 8 | `systemctl daemon-reload && reboot` | Reload generated mount units, then reboot to confirm. |

## COMMON ROOT CAUSES

Wrong or missing UUID in fstab, a removed or renamed disk or network share, filesystem corruption, a failed fsck, an NFS mount without nofail, kernel or initramfs problems after an update.

## FIX AND PREVENTION

Correct fstab, use UUIDs, add `nofail` for secondary mounts, and always run `mount -a` after editing. Keep recent snapshots. For cloud VMs use the serial console and a rescue instance.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Why use UUIDs in fstab?**

A. Device names like /dev/sdb can change between boots. UUIDs identify the filesystem reliably.

**Q. What does nofail do?**

A. Boot continues even if that mount is unavailable, instead of dropping into emergency mode.

## AVOID THIS MISTAKE

Editing fstab and rebooting without testing it using mount -a.

# 09 HIGH CPU BUT TOP SHOWS NOTHING

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**The CPU is very busy on a server, but no single process stands out in top. What could be happening and how do you find it?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

If the CPU is busy but no process stands out, I suspect short-lived processes, such as a script or cron job that forks thousands of children, or time spent in the kernel, interrupts or soft IRQs. First I look at the CPU breakdown with `mpstat`: user, system, iowait, irq, softirq and steal. Then I look for churn: very high context switches in `vmstat`, `ps` sorted by start time, or `execsnoop` from bcc-tools to see every new process as it starts. If system or softirq time is high, I check network interrupts and packet rates. If steal is high, the hypervisor is the cause. Once I find the parent, I trace what schedules it, usually cron, a timer or a monitoring agent.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `mpstat -P ALL 1` | Per-CPU %usr, %sys, %iowait, %irq, %soft and %steal show where the time goes. |
| 2 | `vmstat 1` | Check `cs` (context switches) and `in` (interrupts). Very high values mean process churn. |
| 3 | `pidstat -u 1` | Per-process CPU. Short-lived children are missed because they die between samples. |
| 4 | `ps -eo pid,ppid,etimes,cmd --sort=etimes \| head -20` | The newest processes. Many tiny ones with the same parent means a fork storm. |
| 5 | `sudo execsnoop-bpfcc` | (bcc-tools) Live trace of every new process that starts. |
| 6 | `sudo perf top` | Where CPU time goes, including kernel functions. |
| 7 | `cat /proc/interrupts and sar -n DEV 1` | IRQ distribution and packet rate for network-driven load. |
| 8 | `systemctl list-timers and crontab -l` | Which scheduled job spawns the processes? |

## COMMON ROOT CAUSES

A script or cron job forking constantly (a loop, health check or monitoring agent), kernel or driver time, softirq from heavy network traffic, steal time on shared VMs, container CPU throttling, malware.

## FIX AND PREVENTION

Fix or rate-limit the looping job, move the work into a long-running process, tune interrupt affinity or NIC settings, resize the instance, and add alerts for context switches and process creation rate.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. What does %steal mean?**

A. Time the VM wanted to run but the hypervisor ran something else. High steal means noisy neighbours or an undersized instance.

**Q. Why can top miss it?**

A. top samples at intervals. Processes that start and exit between samples are never seen.

## AVOID THIS MISTAKE

Restarting the server. You lose the pattern you need to catch.

# 10 DATABASE: TOO MANY CONNECTIONS

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**Your application starts failing with 'too many connections' from the database. What do you do now, and how do you prevent it?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

First I stabilise: compare the connection count with `max_connections`, see who holds the connections, and terminate idle or runaway sessions. Then I find the reason. Common causes are a connection leak in the app, pools configured too large and multiplied across many replicas, long transactions or slow queries holding connections, or a traffic spike after a deployment. For the long term I'd right-size the pools: replicas times pool size must stay below the database limit. I'd add a connection pooler such as PgBouncer, ProxySQL or RDS Proxy, set timeouts for idle sessions and long transactions, and alert at 80% of max connections.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `SHOW STATUS LIKE 'Threads_connected'; (MySQL)` | Current connections compared with `max_connections`. |
| 2 | `SELECT count(*), state FROM pg_stat_activity GROUP BY state; (Postgres)` | How many are active, idle, or idle in transaction. |
| 3 | `SHOW FULL PROCESSLIST; or SELECT * FROM pg_stat_activity;` | Who is connected, from where, running what, and for how long. |
| 4 | `SELECT pg_terminate_backend(pid); or KILL id;` | Free connections held by stuck sessions (carefully). |
| 5 | `ss -tn state established '( dport = :5432 )' \| awk '{print $4}' \| sort \| uniq -c` | Connections per client, seen from the DB server. |
| 6 | `kubectl get deploy -o wide and check the pool size setting` | Replicas times pool size is the real maximum demand. |
| 7 | Slow query log or pg_stat_statements | Slow queries keep connections busy for longer. |
| 8 | App metrics: pool wait time, active versus idle | Distinguishes a leak (never returned) from real load. |

## COMMON ROOT CAUSES

Connection leak, pools too large across many replicas, long transactions, slow queries, autoscaling creating many new app instances, no pooler, max_connections too low for the instance size.

## FIX AND PREVENTION

Add PgBouncer or RDS Proxy, shrink pool sizes, set idle and statement timeouts, fix connection leaks (always close connections), optimise slow queries, and alert on connection saturation.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Why not just raise max_connections?**

A. Each connection costs memory and CPU, and too many hurt performance. A pooler is the better fix.

**Q. What is the pool size rule?**

A. Replicas times pool size must stay under the database limit, leaving headroom for admin and batch jobs.

## AVOID THIS MISTAKE

Raising max_connections as the only fix.

# 11 CLOCK DRIFT BREAKS AUTHENTICATION

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**Logins and API calls suddenly fail with errors like 'token expired' or 'certificate not yet valid' on some servers. What could be the cause?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

That pattern points to clock drift. Almost everything time-sensitive breaks when the clock is wrong: TLS certificate validation, Kerberos, JWT tokens, TOTP codes and AWS request signatures, and logs become impossible to correlate. I'd check `timedatectl` to see whether the clock is synchronised, and `chronyc tracking` for the offset. I'd confirm the time sources are reachable, because UDP port 123 is often blocked, restart chrony, and step the clock if the offset is large. On VMs I'd also check the hypervisor's time sync settings so they don't fight with NTP. Long term: run chrony everywhere, use the cloud provider's time service, and monitor the offset.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `timedatectl` | Shows `System clock synchronized: yes or no` and the timezone. |
| 2 | `chronyc tracking` | Current offset from the reference, stratum and last sync. |
| 3 | `chronyc sources -v` | Which time servers are used, and are they reachable? |
| 4 | `sudo systemctl restart chronyd (or systemd-timesyncd)` | Restart the time sync service. |
| 5 | `sudo chronyc makestep` | Force an immediate correction instead of slow slewing. |
| 6 | `date -u and curl -sI https://example.com \| grep -i ^date` | Compare your UTC time with a remote server's Date header. |
| 7 | `nc -zuv pool.ntp.org 123` | Is outbound NTP (UDP 123) reachable through the firewall? |
| 8 | `sudo hwclock --show` | Hardware clock value, for VMs that start with the wrong time. |

## COMMON ROOT CAUSES

NTP daemon not running, UDP 123 blocked, wrong or unreachable time servers, a VM paused or restored from a snapshot, hypervisor and guest both adjusting time, timezone confusion, a dead CMOS battery on physical servers.

## FIX AND PREVENTION

Install and enable chrony, point it at reliable sources (cloud time service or internal NTP), allow UDP 123, and monitor the offset (for example `node_timex_offset_seconds`), alerting above about 100 ms.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Why does TLS fail with the wrong time?**

A. Certificates have validity windows. If the clock is outside it, the certificate looks expired or not yet valid.

**Q. NTP versus chrony?**

A. Chrony is the modern implementation: faster sync and better on VMs and intermittent connections.

## AVOID THIS MISTAKE

Setting the time by hand with date and leaving it. It will drift again.

# 12 CONTAINER CANNOT REACH THE INTERNET

TROUBLESHOOTING DIFFICULTY ✦✦✦

## THE QUESTION

**A Docker container can't reach the internet or resolve names, but the host can. How do you debug it?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

I narrow it down: is it DNS, routing, or a firewall or proxy? From inside the container I test by IP first with `ping 1.1.1.1`, then by name. If IP works but names fail, it's DNS, so I check `/etc/resolv.conf` and the daemon's DNS settings. If neither works, I check the host: IP forwarding must be on, and Docker's iptables NAT rule (MASQUERADE) must exist. A firewall reset or a custom iptables setup can remove it. A VPN using a subnet that overlaps Docker's default bridge network is another classic. In the cloud, the route table, NAT gateway and security group egress rules matter as well, and corporate proxies need proxy variables.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `docker exec -it c sh -c 'ping -c2 1.1.1.1'` | Does raw IP connectivity work? |
| 2 | `docker exec -it c sh -c 'nslookup example.com'` | If IP works but this fails, it is DNS. |
| 3 | `docker exec c cat /etc/resolv.conf` | Which DNS server does the container use? |
| 4 | `sysctl net.ipv4.ip_forward` | Must be 1 for containers to reach outside. |
| 5 | `sudo iptables -t nat -L POSTROUTING -n -v` | Look for the MASQUERADE rule for the bridge subnet. |
| 6 | `docker network inspect bridge` | Subnet and gateway. Look for overlap with your VPN or LAN. |
| 7 | `ip route and ip addr show docker0` | Host routes and bridge configuration. |
| 8 | **Cloud: route table, NAT gateway, SG and NACL egress** | Private subnets need a NAT gateway to reach the internet. |

## COMMON ROOT CAUSES

DNS misconfiguration, IP forwarding off, iptables or firewalld flushed or conflicting with Docker, subnet overlap with a VPN or corporate network, no NAT gateway or egress rule, a required proxy, MTU mismatch on VPNs.

## FIX AND PREVENTION

Enable forwarding, restart Docker so it recreates its iptables rules, set the daemon's `dns` and `bip` options, fix the cloud route or egress rule, and configure proxy variables if needed.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. How does Docker NAT work?**

A. Docker adds a MASQUERADE rule so container traffic leaves with the host's IP address.

**Q. Why can a VPN break Docker networking?**

A. The VPN may use a subnet that overlaps docker0's default 172.17.0.0/16, so routes conflict. Change `bip` in the daemon config.

## AVOID THIS MISTAKE

Disabling the firewall to test. Check the specific rules instead.

# 13 HUGE DOCKER IMAGE AND SLOW BUILDS

DEVOPS & CLOUD DIFFICULTY ✦✦✦

**THE QUESTION**

**Your Docker image is 2 GB and every build takes 10 minutes. How do you improve it?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

Large images and slow builds usually come from poor layering and cache use. For builds, I put what changes least first, like installing dependencies, before copying the application source, so Docker's layer cache is reused. I add a `.dockerignore` to keep `.git`, `node_modules` and build output out of the build context. For image size, I use multi-stage builds so compilers and build tools stay in the build stage, choose a slim or distroless base, combine package install and cleanup in a single RUN, and copy only what's needed. In CI I use BuildKit cache mounts or a registry cache. I verify results with `docker history` and a tool like dive.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `docker images and docker history image` | Image size and the size of each layer. |
| 2 | `dive image` | Interactive layer explorer to find wasted space. |
| 3 | `cat .dockerignore` | Are .git, node_modules, tmp and build output excluded from the context? |
| 4 | `DOCKER_BUILDKIT=1 docker build --progress=plain .` | Modern builder: parallel stages and readable timing per step. |
| 5 | `RUN --mount=type=cache,target=/root/.cache/pip pip install -r r.txt` | BuildKit cache mount keeps package caches between builds. |
| 6 | `docker build --cache-from registry/app:cache .` | Reuse layers from a registry in CI. |
| 7 | Multi-stage build | Build in one stage, copy only the binary or artefacts into a slim runtime stage. |
| 8 | Choose the base image | alpine, slim or distroless. Note that alpine uses musl, which can cause compatibility issues. |

**COMMON ROOT CAUSES**

`COPY . .` before installing dependencies (cache invalidated on every change), no .dockerignore, build tools left in the final image, a large base image, separate RUN layers leaving caches behind, no layer cache in CI.

**FIX AND PREVENTION**

Order the Dockerfile from least to most frequently changed, use multi-stage builds, clean caches in the same RUN, pin versions, and scan with Trivy. Aim for small, reproducible, non-root images.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. Why does layer order matter?**

A. Docker reuses a layer only if it and all earlier layers are unchanged. A change early on invalidates everything after it.

**Q. Alpine versus distroless?**

A. Alpine is small, with a shell and package manager (musl libc). Distroless has no shell or package manager, which reduces attack surface but is harder to debug.

**AVOID THIS MISTAKE**

Copying the whole repository before installing dependencies.

# 14 CI PIPELINE IS SLOW AND FLAKY

DEVOPS & CLOUD DIFFICULTY ✦✦✦

**THE QUESTION**

**Your CI pipeline has become slow, and some tests fail randomly. How do you improve it?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

I'd work from data, not guesses. First I look at pipeline duration and failure trends per stage and per job to find where time goes and which tests fail intermittently. Slowness usually comes from missing dependency or Docker caches, serial jobs that could run in parallel, a big test suite, under-powered runners, or waiting in queues. Flakiness comes from shared state, timing assumptions, network dependencies, resource contention or unpinned dependencies. My fixes are: cache dependencies and layers, parallelise and shard tests, run only affected tests on pull requests, use larger or dedicated runners, containerise external services, pin versions, and quarantine flaky tests with an owner and a ticket while fixing them.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `gh run list --limit 20 and gh run view ID --log` | Durations and failures across recent runs. |
| 2 | **Timing per step in the CI UI** | Where is the time: install, build, test, deploy, or waiting for a runner? |
| 3 | **Cache dependencies** | actions/cache, GitLab cache or Docker layer cache for node_modules, pip, Maven and Go modules. |
| 4 | **Parallelise** | Matrix builds, test sharding (pytest-xdist, jest --shard) and independent jobs in parallel. |
| 5 | **Run less on pull requests** | Path filters and affected-only tests. Run the full suite on main or nightly. |
| 6 | **Detect flakiness** | Re-run failed tests, track failure rate per test, quarantine with owners. |
| 7 | `docker stats on the runner` | Are runners starved of CPU or memory? |
| 8 | **Pin versions and use lockfiles** | Unpinned dependencies or images cause 'worked yesterday' failures. |

**COMMON ROOT CAUSES**

No caching, serial jobs, one huge test stage, small shared runners, queue time, tests that depend on time, order, shared databases or external APIs, unpinned dependencies, resource contention.

**FIX AND PREVENTION**

Cache, parallelise, right-size runners, isolate tests with ephemeral containers, fail fast with lint and unit tests first, and track pipeline metrics (duration, failure rate, flaky rate) as a team goal.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. How do you handle a flaky test?**

A. Reproduce it and find the shared state or timing cause, then fix it. Meanwhile quarantine it with an owner so it doesn't block everyone.

**Q. Why fail fast?**

A. Cheap checks (lint, unit tests) first give feedback in minutes and save runner time.

**AVOID THIS MISTAKE**

Auto-retrying everything. It hides real problems and wastes time.

# 15 TERRAFORM STATE LOCK OR LOST STATE

DEVOPS & CLOUD DIFFICULTY ✦✦✦

**THE QUESTION**

**terraform apply fails with 'Error acquiring the state lock', and a colleague says the state may be corrupted or lost. What do you do?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

For a lock error, I first confirm nobody is actually running Terraform: I read the lock info (who, when, which operation) and check whether a CI job is still running. If the run crashed, the lock is stale and `terraform force-unlock` is appropriate, but only after I've confirmed nobody is applying. For lost or damaged state, I don't panic: if the backend is S3 with versioning, I restore the previous state version. Otherwise I rebuild by importing existing resources, with `terraform import` or import blocks, and compare with `plan` until there is no unexpected diff. To prevent this: a remote backend with locking and versioning, encryption, restricted access, applies only from CI, and never editing state by hand.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `terraform plan` | Shows the lock error: lock ID, who, operation and when it was created. |
| 2 | Check CI and teammates | Is another pipeline or person applying right now? Never unlock a live run. |
| 3 | `terraform force-unlock LOCK_ID` | Release a stale lock once you are sure nobody is applying. |
| 4 | `aws s3api list-object-versions --bucket b --prefix path/terraform.tfstate` | List state versions in a versioned S3 backend. |
| 5 | `aws s3api get-object --bucket b --key path/terraform.tfstate --version-id V out.tfstate` | Download a known-good earlier version. |
| 6 | `terraform state pull > backup.tfstate` | Always back up the current state before any surgery. |
| 7 | `terraform import aws_instance.web i-123 (or import blocks)` | Rebuild missing state entries from real resources. |
| 8 | `terraform plan (repeat)` | Continue until the plan shows no unexpected changes. |

**KEY POINTS TO MENTION**

State maps your code to real resources. Locking (S3 with DynamoDB, native S3 locking in newer versions, or Terraform Cloud) prevents concurrent applies. Versioning gives you a restore path. State can contain secrets, so encrypt it and restrict access.

**FIX AND PREVENTION**

Use a remote backend with locking, versioning and encryption. Keep one state per environment or component, apply only from CI, use least-privilege access, and never commit state to Git.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. Why not commit state to Git?**
A. It can contain secrets, has no locking, and merges badly.

**Q. What are import blocks?**
A. A declarative way (Terraform 1.5 and later) to import existing resources, reviewable in a plan.

**AVOID THIS MISTAKE**

Running force-unlock while someone is mid-apply. That can corrupt state.

# 16 A SECRET LEAKED INTO GIT

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**A developer accidentally committed an AWS access key to a repository. What do you do?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

I treat any secret in Git as compromised, even if I delete it in the next commit, because it stays in history and may already be cloned or scraped. Step one is to revoke and rotate the credential immediately. Step two is to check for misuse: CloudTrail or the service's audit logs for activity using that key. Then I remove it from history if needed using `git filter-repo` or BFG and force-push, and tell collaborators to re-clone, but rotation is what actually removes the risk. After that I prevent a repeat with pre-commit secret scanning such as gitleaks, CI checks, short-lived credentials like IAM roles and OIDC, and a secrets manager such as AWS Secrets Manager or Vault.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | Revoke and rotate the secret first | Disable the key or password and issue a new one. This is the real fix. |
| 2 | `aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId, AttributeValue=AKIA...` | See what the leaked key did, and from where. |
| 3 | `gitleaks detect --source . --log-opts='--all'` | Scan the whole history for other secrets. |
| 4 | `git filter-repo --replace-text secrets.txt` | Rewrite history to remove the secret (or use BFG Repo-Cleaner). |
| 5 | `git push --force --all and git push --force --tags` | Publish the cleaned history, then ask everyone to re-clone. |
| 6 | Check forks, CI logs and artefacts | Secrets can be copied into forks, build logs, caches and container images. |
| 7 | `pre-commit install (with a gitleaks hook)` | Block secrets before they are committed. |
| 8 | Move secrets into a manager | Secrets Manager, Vault, SSM Parameter Store or external-secrets, injected at runtime. |

## KEY POINTS TO MENTION

Rotation beats deletion. Rewriting history alone isn't enough because the secret may already be copied. Check access logs for misuse, follow the incident process, and inform the owner and the security team.

## FIX AND PREVENTION

Use short-lived credentials (IAM roles, OIDC from CI) instead of static keys, secret scanning in pre-commit and CI, `.gitignore` for env files, scoped least-privilege keys, and secret manager integration.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Why isn't deleting the commit enough?**

A. History, forks and clones still contain it, and automated scanners find leaked keys within minutes.

**Q. What is OIDC for CI?**

A. The CI job gets short-lived cloud credentials by proving its identity, so no long-lived keys are stored.

## AVOID THIS MISTAKE

Quietly deleting the file and not rotating the secret.

# 17 AWS BILL SUDDENLY DOUBLED

DEVOPS & CLOUD DIFFICULTY ✦✦✦

**THE QUESTION**

**Your AWS bill has doubled this month. How do you find the cause and stop it from happening again?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

I treat it like an incident: find what changed, who owns it, stop the bleeding, then prevent a repeat. In Cost Explorer I group by service, then usage type, account, region and tag to find the spike and its start date, and I compare that with deployments and changes around then. Common culprits are NAT gateway data processing, cross-AZ and internet data transfer, forgotten or oversized EC2 and RDS instances, unattached EBS volumes and snapshots, S3 requests and missing lifecycle rules, and runaway Lambda or logging costs. CloudTrail tells me who created resources. Fixes include deleting or rightsizing, lifecycle policies, VPC endpoints, savings plans, and budgets with anomaly detection so we get alerted early.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `aws ce get-cost-and-usage --time-period Start=2026-10-01,End=2026-11-01 --granularity DAILY --metrics UnblendedCost --group-by Type=DIMENSION,Key=SERVICE` | Daily cost by service from the CLI. |
| 2 | Cost Explorer: group by Usage type, Region, Account, Tag | Pinpoint exactly what grew and when it started. |
| 3 | `aws ec2 describe-volumes --filters Name=status,Values=available` | Unattached EBS volumes still billing. |
| 4 | `aws ec2 describe-addresses --query 'Addresses[?AssociationId==`null`]'` | Unused Elastic IPs. |
| 5 | `aws ec2 describe-nat-gateways plus CloudWatch BytesOutToDestination` | NAT data processing is a classic surprise cost. |
| 6 | `aws s3api list-buckets and S3 Storage Lens` | Which buckets grew? Are lifecycle rules missing? |
| 7 | `aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=RunInstances` | Who launched resources, and when? |
| 8 | AWS Budgets and Cost Anomaly Detection | Alerts for forecasted overspend and unusual daily changes. |

**COMMON ROOT CAUSES**

Forgotten test resources, oversized instances, autoscaling gone wild, NAT gateway traffic, cross-AZ and internet data transfer, snapshots and logs without retention, high S3 request volume, a compromised account mining crypto, data egress from a new feature.

**FIX AND PREVENTION**

Remove or rightsize waste, add lifecycle and retention policies, use VPC endpoints, buy Savings Plans or reserved capacity for steady workloads, schedule non-production shutdowns, and enforce tagging with budgets per team.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. What is FinOps?**

A. A practice of making cloud cost visible and owned by teams, using tagging, budgets and regular reviews.

**Q. How do you reduce NAT gateway cost?**

A. Use gateway endpoints for S3 and DynamoDB, interface endpoints for other services, and keep traffic in-region and in-AZ.

**AVOID THIS MISTAKE**

Deleting resources before checking ownership and backups.

# 18 PATCH 500 SERVERS SAFELY

ADMIN & SECURITY DIFFICULTY ✦✦✦

## THE QUESTION

**A critical vulnerability needs patching across 500 Linux servers. How do you do it without causing an outage?**

## SAY IT LIKE THIS (60-90 seconds, out loud)

I'd make patching a repeatable, automated, staged process. First I classify servers by criticality and group them, so we never patch both nodes of a redundant pair at once. I patch in rings: a test ring first, then staging, then a small production canary, then the rest in batches, with health checks and an automatic stop on failure. Automation comes from Ansible or AWS Systems Manager Patch Manager, using maintenance windows. Before each batch I confirm backups or snapshots, drain nodes from the load balancer, patch, reboot if the kernel changed, verify services and metrics, and return the node. For urgent CVEs I use an expedited path. Immutable infrastructure, where we build a patched image and replace servers, is even safer.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `apt list --upgradable or dnf check-update` | What updates are pending, including security ones? |
| 2 | `sudo unattended-upgrade --dry-run -d or dnf updateinfo list security` | Preview security-only updates. |
| 3 | `ansible-playbook patch.yml -l ring1 --check --diff` | Dry-run the patch playbook on the first ring. |
| 4 | `ansible-playbook patch.yml -l canary --forks 5` | Patch a small canary group, then verify. |
| 5 | `ansible-playbook patch.yml -l prod (with serial: 10%)` | Rolling batches, stopping on failure (`serial` and `max_fail_percentage`). |
| 6 | `needs-restarting -r (RHEL) or cat /var/run/reboot-required` | Is a reboot required? |
| 7 | `uname -r and systemctl --failed` | Verify the new kernel and that services came back. |
| 8 | Systems Manager Patch Manager and maintenance windows | Cloud-native option with baselines and compliance reports. |

## KEY POINTS TO MENTION

Rings and batches limit blast radius. Always have backups or snapshots and health checks. Keep an inventory and compliance reporting. Have an expedited path for critical CVEs and a rollback plan.

## FIX AND PREVENTION

Automate with Ansible or SSM, patch regularly (for example monthly) so the delta stays small, use golden images where possible, and track compliance and failures on a dashboard.

## FOLLOW-UP QUESTIONS THEY MAY ASK

**Q. Patch in place or replace servers?**

A. Replacing from a freshly built, tested image (immutable) is more consistent and rolls back more easily. In-place patching suits long-lived servers.

**Q. How do you handle kernel updates?**

A. Schedule reboots in rolling batches, or use live patching (kpatch, Canonical Livepatch) for critical fixes.

## AVOID THIS MISTAKE

Patching everything at once, or skipping the test ring.

# 19 BACKUP AND DISASTER RECOVERY PLAN

ADMIN & SECURITY DIFFICULTY ✦✦✦

**THE QUESTION**

**How would you design a backup and disaster recovery strategy for a production application?**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

I start with the business requirements: RPO is how much data we can afford to lose, and RTO is how long we can be down. Those decide the design and the cost. For backups I follow the 3-2-1 rule: three copies, on two different media, with one off-site, ideally in another region or account and immutable against ransomware. I'd back up databases with point-in-time recovery, volumes with snapshots, and keep infrastructure and configuration as code so everything can be recreated. Most importantly, I'd test restores regularly, because an untested backup is only a hope. For DR I pick a strategy that matches the RTO: backup and restore, pilot light, warm standby or active-active, and I document runbooks and run game days.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | `aws ec2 create-snapshot --volume-id vol-123 or AWS Backup plans` | Scheduled, centrally managed snapshots with retention. |
| 2 | `pg_dump -Fc db > db.dump and WAL archiving or RDS PITR` | Logical dumps plus continuous archiving for point-in-time recovery. |
| 3 | `restic backup /data or borg create ...` | Efficient, encrypted, incremental file backups. |
| 4 | `aws s3api put-bucket-versioning and S3 Object Lock` | Versioning and immutability protect against deletion and ransomware. |
| 5 | `aws s3 sync s3://src s3://dr-bucket --source-region r1 --region r2` | Copy data to another region (or use cross-region replication). |
| 6 | `restic restore latest --target /restore-test` | Restore test: prove you can actually recover. |
| 7 | `terraform apply (in the DR region)` | Recreate infrastructure from code during a disaster. |
| 8 | Runbooks and game days | Document steps, owners and contacts. Practise failover regularly. |

**KEY POINTS TO MENTION**

RPO and RTO drive the design. Use the 3-2-1 rule and immutability. Test restores and measure the real recovery time. Back up configuration, secrets (securely) and DNS, not only data. Monitor backup jobs and alert on failures.

**FIX AND PREVENTION**

Choose a DR tier that matches business needs and budget, automate backups and infrastructure creation, keep copies in a separate account and region, encrypt everything, and test at least quarterly.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. RPO versus RTO?**

A. RPO: maximum acceptable data loss, measured in time. RTO: maximum acceptable downtime.

**Q. Backup versus replication?**

A. Replication copies changes, including mistakes and deletions, almost immediately. Backups give point-in-time history. You need both, for different failures.

**AVOID THIS MISTAKE**

Never testing restores.

# 20 DESIGN A HIGHLY AVAILABLE WEB APP

DEVOPS & CLOUD DIFFICULTY ✦✦✦

**THE QUESTION**

**Design a highly available, secure web application on AWS that survives an availability zone failure and handles traffic spikes.**

**SAY IT LIKE THIS (60-90 seconds, out loud)**

I'd spread every tier across at least two, ideally three, availability zones. Users reach the app through Route 53 and CloudFront, then an Application Load Balancer in public subnets. The application runs in an Auto Scaling group or EKS in private subnets across the AZs, scaling on CPU or request metrics, with health checks replacing bad instances. The database is a managed Multi-AZ service such as RDS or Aurora in isolated private subnets, with read replicas if needed. Static content goes to S3 and CloudFront, and ElastiCache reduces database load. For security I use least-privilege security groups, WAF, TLS everywhere, IAM roles instead of keys, secrets in Secrets Manager, and logging with CloudTrail and CloudWatch. It's all defined in Terraform and deployed through CI/CD.

| # | COMMAND / CHECK | WHAT IT TELLS YOU |
|---|---|---|
| 1 | **Edge: Route 53, CloudFront, WAF, ACM certificates** | DNS, caching, attack filtering and managed TLS. |
| 2 | **Load balancing: ALB across 2 to 3 AZs** | Health checks and path-based routing. Only the ALB sits in public subnets. |
| 3 | **Compute: Auto Scaling group or EKS in private subnets** | Scale on metrics, replace unhealthy nodes, spread across AZs, launch from a golden AMI or image. |
| 4 | **Data: RDS or Aurora Multi-AZ, automated backups, PITR** | Automatic failover. Add read replicas and ElastiCache to offload the database. |
| 5 | **Storage: S3 for static files and uploads** | Durable, versioned and encrypted at rest. |
| 6 | **Network: VPC with public, private and isolated subnets, NAT per AZ, VPC endpoints** | Private tiers with no direct internet access to app or database. |
| 7 | **Security: IAM roles, SGs, Secrets Manager, KMS, CloudTrail, GuardDuty** | Least privilege, encryption and detection. |
| 8 | **Operations: CloudWatch, alarms, Terraform, CI/CD** | Observability, repeatable deployments and rollbacks. |

**KEY POINTS TO MENTION**

No single point of failure: every tier is multi-AZ. The application tier is stateless, so it can scale and be replaced. Managed services reduce operational work. Mention trade-offs: cost (a NAT per AZ, Multi-AZ database), complexity, and multi-region only if RTO and RPO demand it.

**FIX AND PREVENTION**

Test it: kill an instance, fail over the database, and run a load test. Set SLOs and alerts, review the Well-Architected pillars (reliability, security, cost, performance, operations, sustainability), and write the runbook.

**FOLLOW-UP QUESTIONS THEY MAY ASK**

**Q. How do you survive a whole region failure?**

A. Multi-region: replicate data (Aurora Global Database, S3 cross-region replication), keep a warm standby or active-active, and use Route 53 failover. It costs more, so only if the RTO and RPO require it.

**Q. How do you handle sudden traffic spikes?**

A. Auto Scaling with target tracking, caching at CloudFront and ElastiCache, queues to smooth bursts, and load testing before launch.

**AVOID THIS MISTAKE**

Putting the database in a public subnet, or running everything in one AZ.

## 21 RAPID-FIRE ROUND

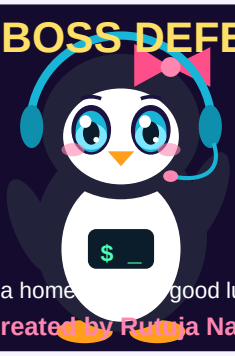
*10 quick questions, 1 to 2 sentence answers*

| | | |
|---|---|---|
| 1 | **What are the four golden signals?** | Latency, traffic, errors and saturation. Monitor these for every service. |
| 2 | **RPO versus RTO?** | RPO: how much data you can lose. RTO: how long you can be down. |
| 3 | **What is a bastion host?** | A hardened jump server in a public subnet that is the only way in to private servers over SSH. Often replaced by SSM Session Manager. |
| 4 | **Security group versus NACL?** | Security group: stateful, allow-only, per instance. NACL: stateless, allow and deny, per subnet, ordered rules. |
| 5 | **A record versus CNAME?** | A maps a name to an IP address. CNAME makes a name an alias of another name (not allowed at the zone apex). |
| 6 | **StatefulSet versus Deployment?** | StatefulSet gives pods stable names, ordered rollout and their own persistent storage. Deployment is for stateless, interchangeable pods. |
| 7 | **What is immutable infrastructure?** | You never patch servers in place. You build a new image and replace the old servers. |
| 8 | **What is a service mesh?** | Sidecar proxies that handle mTLS, retries, traffic splitting and telemetry between services, for example Istio or Linkerd. |
| 9 | **SLI, SLO and SLA?** | SLI: a measurement (such as success rate). SLO: the internal target for it. SLA: the contract with penalties if you miss it. |
| 10 | **Forward proxy versus reverse proxy?** | A forward proxy sits in front of clients and hides them. A reverse proxy sits in front of servers (Nginx, HAProxy) and routes or protects them. |

**BEFORE THE INTERVIEW: FINAL CHECKLIST**

- Practise each answer out loud in 60 to 90 seconds. Record yourself once.
- Prepare a real example for each topic: "Last time I saw this, I..."
- Know why a command works, not just its syntax.
- Admit what you don't know, then explain how you would find out.
- End answers with prevention: monitoring, automation, documentation.
- Keep a home lab (VM, Docker, minikube). Break things on purpose and fix them.

## FINAL BOSS DEFEATED!




Practice out loud, build a home[illegible] good luck with your interviews.

**Created by Rutuja Naik**